DEEP Q-LEARNING

INTERMEDIATE DEEP LEARNING

Zhuofan XU
AI / Reinforcement Learning Engineer @ Deepmath
PhD in AI - Université Paris-Saclay · ENS Paris-Saclay

RL AND DRL

DEEP REINFORCEMENT LEARNING (DRL)

- To address complex and high dimensional problems
- Neural networks as policy function or value function
- Example:
  - Tabular Q Learning → Deep Q Network (DQN)
- Better generalization

Q-Table	a_1	a_2	a_3	...	a_m
s_1					
s_2					
s_3					
s_4					
...					
s_n					

Figure : Q Table

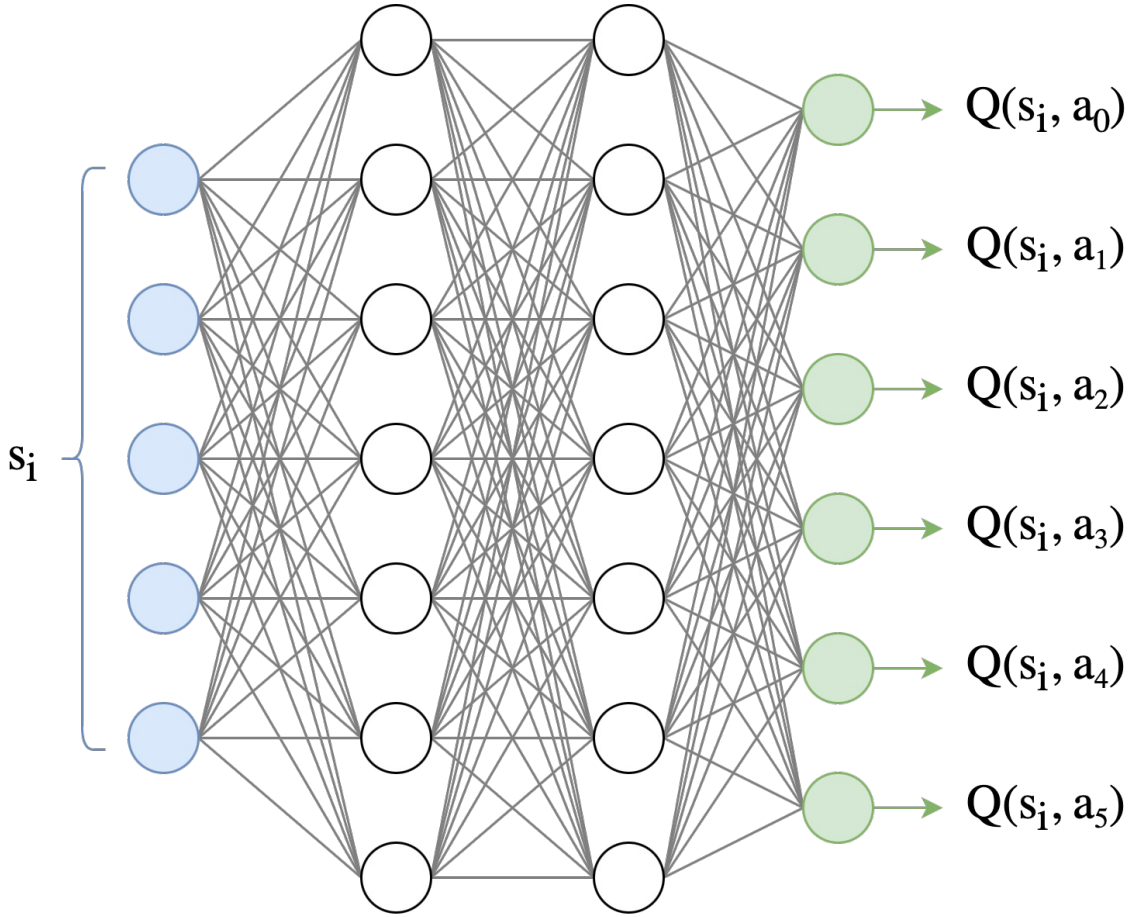


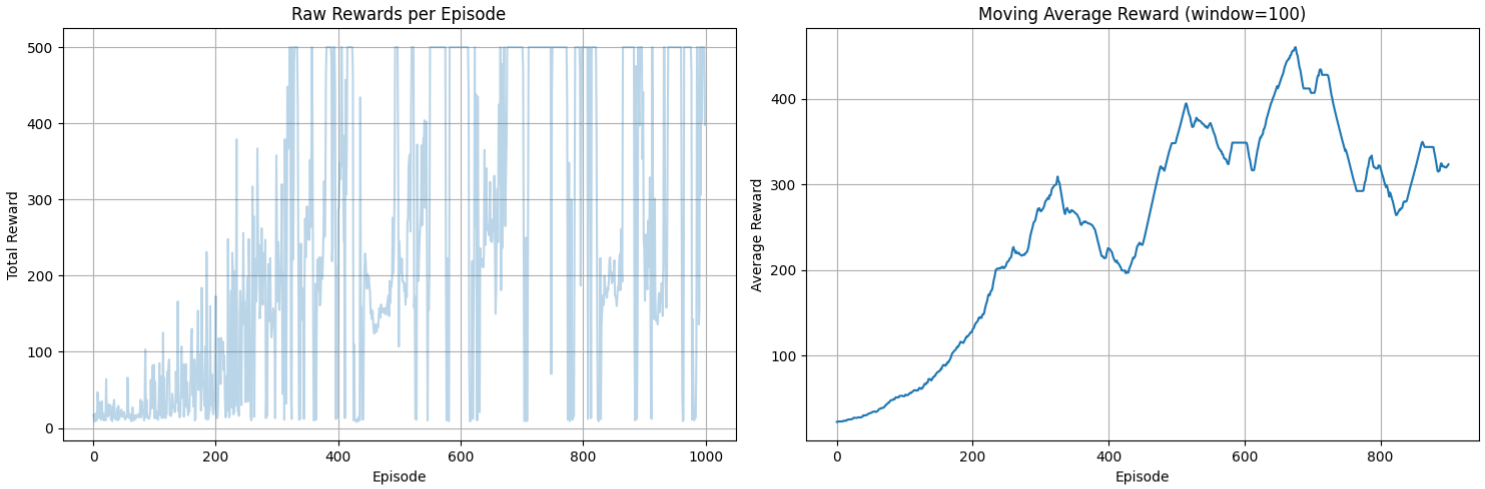
Figure : Q Network

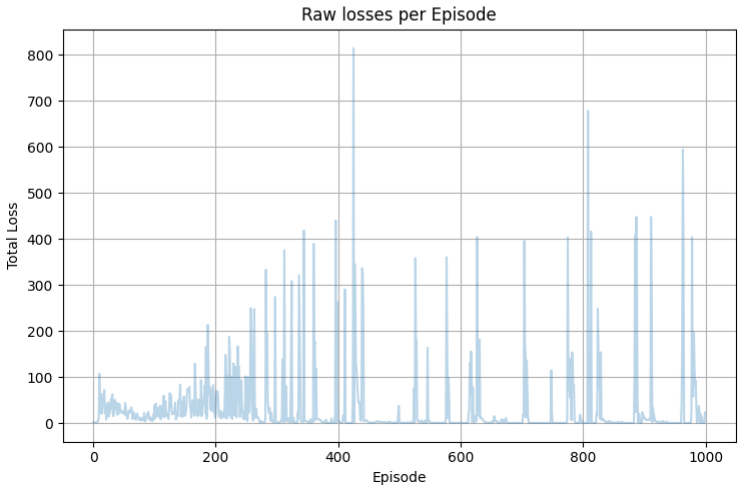
ISSUES

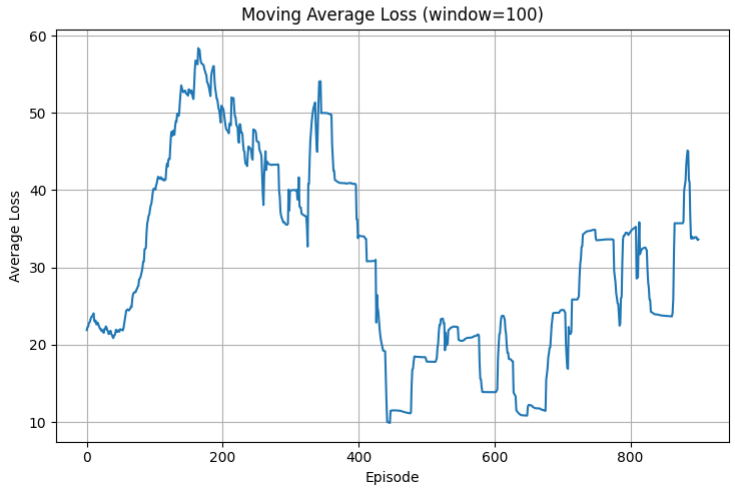
PROBLEMS WITH SIMPLE Q-NETWORK

CONTEXT

PROBLEMS WITH SIMPLE Q-NETWORK







- Instability of performance
  - Even when ϵ reached 0.01
- Model did not converge

CONTEXT

POTENTIAL REASONS

- Source of instability from the update rule?

$$\mathcal{L}(\theta) = \left[r + \gamma \max_{a'} Q_\theta\,(s', a') - Q_\theta(s, a)\right]^2$$

  - θ denotes the parameters of the Q-network

- From the implementation?

CONTEXT

POTENTIAL REASONS

- From the update rule:

$$\mathcal{L}(\theta) = \left[r + \gamma \max_{a'} Q_\theta\,(s', a') - Q_\theta(s, a)\right]^2$$

  - Target and current Q-values share the same parameters (θ) → moving target during training
- From the implementation
  - Batch size = 1 → high gradient noise
  - Experiences discarded after one update → Poor sample efficiency, catastrophic forgetting

SOLUTION

TARGET Q-NETWORK – EXPERIENCE REPLAY

SOLUTION

TARGET Q-NETWORK

- To "fix" the target during training, we introduce an additional target Q-network, whose parameters (θ^-) are delayed compared to the online Q-network (θ).
- Original loss:

$$\mathcal{L}(\theta) = \left[r + \gamma \max_{a'} Q_{\theta}(s', a') - Q_{\theta}(s, a)\right]^2$$

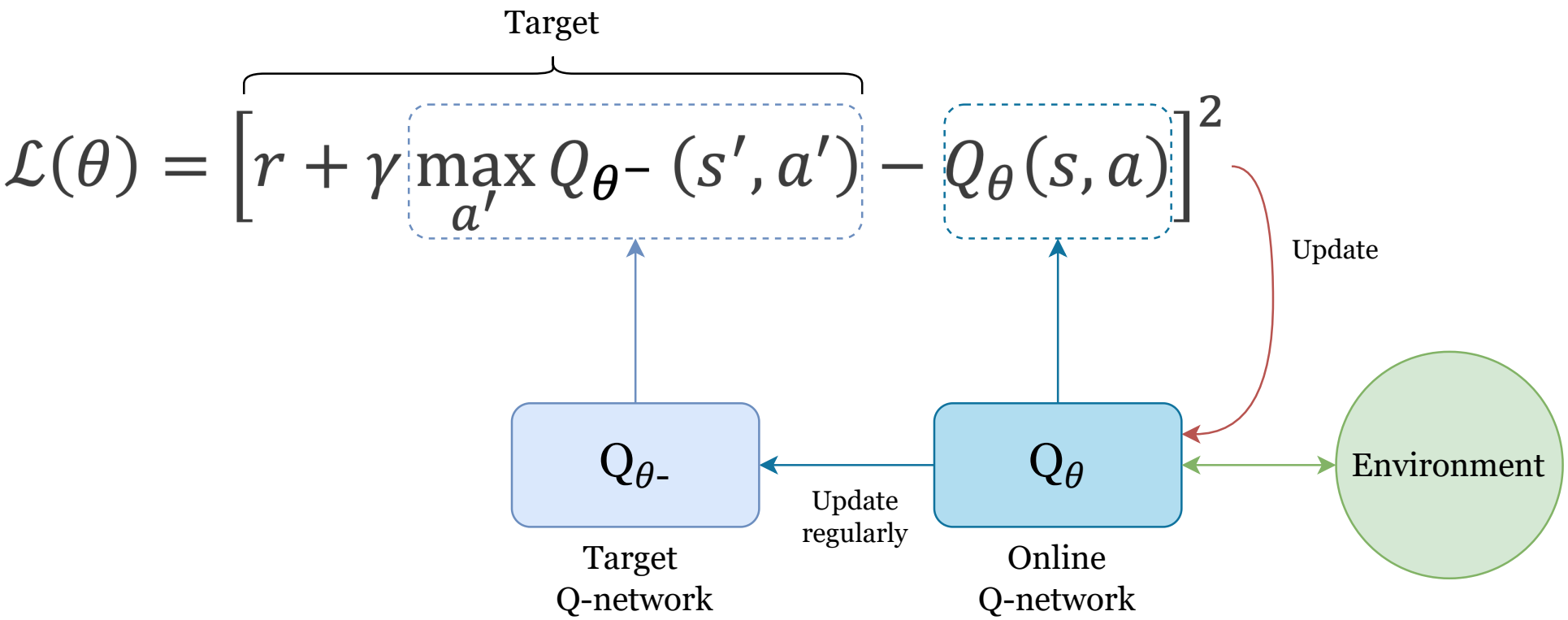
- Fixed-target loss:

$$\mathcal{L}(\theta) = \left[r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_{\theta}(s, a)\right]^2$$

- The target Q-network is periodically synchronized with the online Q-network after a fixed number of updates.

SOLUTION

TARGET Q-NETWORK



Deep Q-learning with target network

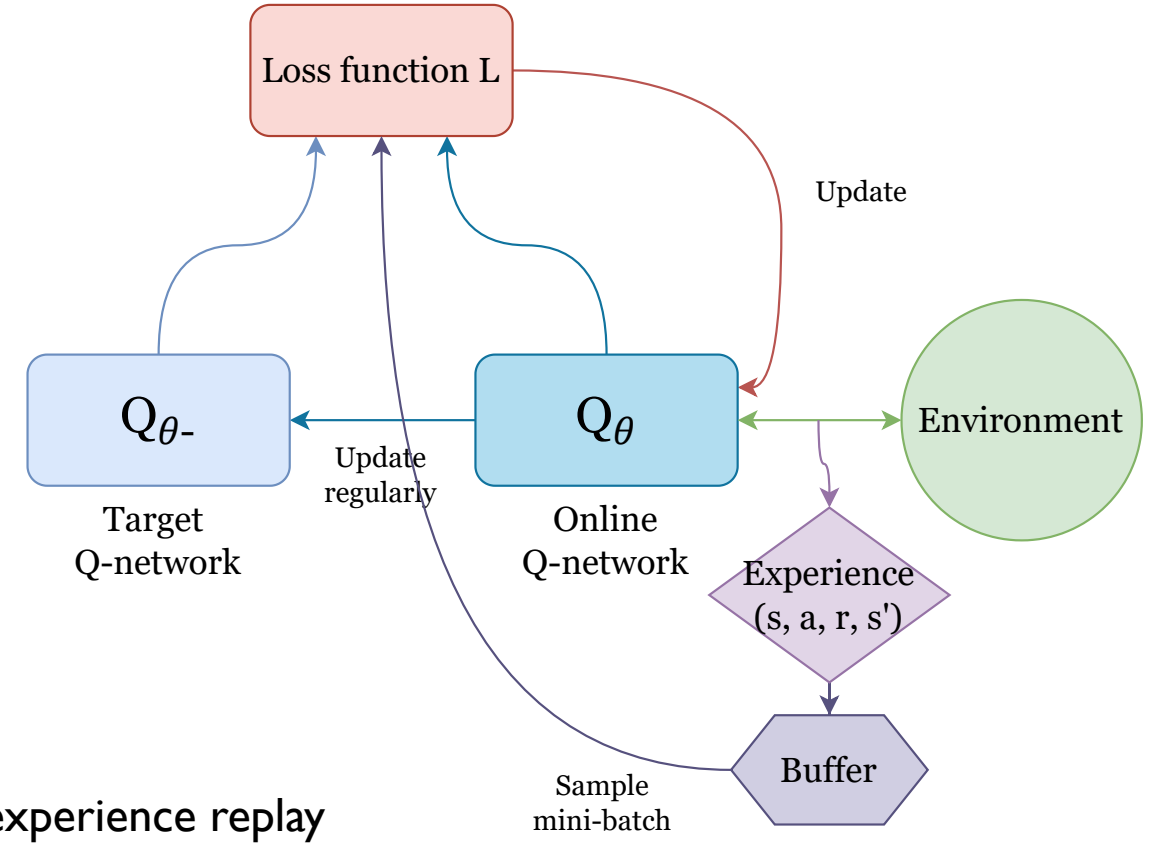
SOLUTION

EXPERIENCE REPLAY

- A buffer is used to stock experiences $(s, a, r, s', \text{done})$
- Enables formation of mini-batches for updates
- When buffer size $>$ batch size $\rightarrow$ sample from the buffer for training
- Advantages:
  - Higher data efficiency
  - Reduces forgetting
  - Breaks temporal correlations between samples (SGD prefers i.i.d. data)

SOLUTION

EXPERIENCE REPLAY



Deep Q-learning with target network and experience replay

FURTHER ENHANCEMENTS

DOUBLE DQN – PRIORITIZED EXPERIENCE REPLAY

FURTHER ENHANCEMENTS

DOUBLE DQN

- DQN tends to overestimate the Q-values
- Target: $y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$
- Q-values are noisy estimate
- → If a Q-value has a positive noise, it is more likely to be selected for the target calculation.
- → Leads to a systematic overestimation bias.

FURTHER ENHANCEMENTS

DOUBLE DQN

- Solution: Decouple the action selection and the evaluation
  - Use the online network Q_θ to select the best action
  - Use the target network Q_{θ^-} to evaluate that action
- Original target:

$$y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$$

- New target:

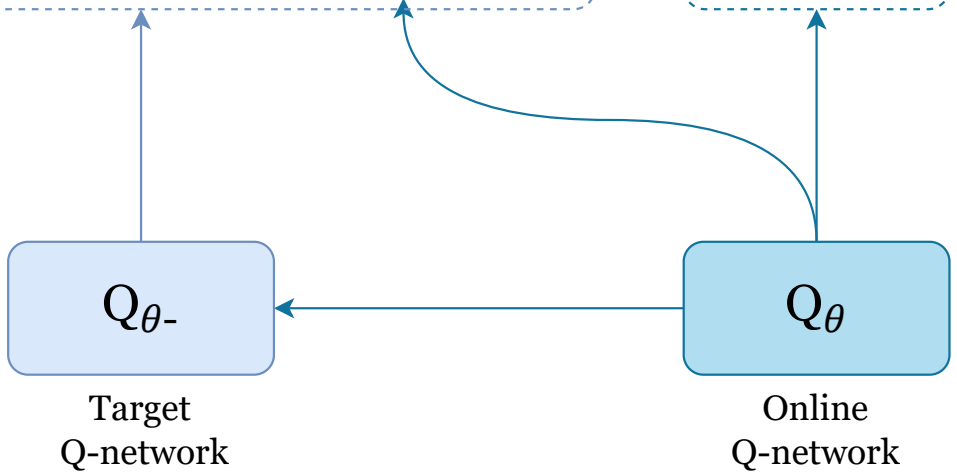
$$y = r + \gamma Q_{\theta^-}\left(s', \arg\max_{a'} Q_\theta(s', a')\right)$$

- Since the two networks have uncorrelated errors, the resulting target is less biased and more stable.

FURTHER ENHANCEMENTS

DOUBLE DQN

$$\mathcal{L}(\theta) = \left[r + \gamma Q_{\theta^-}\left(s', \arg\max_{a'} Q_\theta(s', a')\right) - Q_\theta(s, a)\right]^2$$



Loss function of DDQN

FURTHER ENHANCEMENTS

PRIORITIZED EXPERIENCE REPLAY

- In standard Experience Replay, experiences $(s, a, r, s', \text{done})$ are sampled uniformly from the replay buffer
- However, not all experiences are equally useful for learning.
  - Transitions with small TD errors → the agent already predicts them well.
  - Transitions with large TD errors → the agent can learn more from them.
- To improve data efficiency, we can sample transitions with large TD errors more often

FURTHER ENHANCEMENTS

PRIORITIZED EXPERIENCE REPLAY

- Prioritized experience replay makes the sampling probability of experiences proportional to their TD error magnitude

$$P(i) = \frac{|\delta_i|^\alpha}{\sum_k |\delta_k|^\alpha}$$

  - Where $\alpha \in [0,1]$ (not the learning rate before): prioritization strength
- $\rightarrow$ agent learns more often from informative or surprising transitions, speeding up convergence

FURTHER ENHANCEMENTS

PRIORITIZED EXPERIENCE REPLAY

- However, non-uniform sampling introduces bias, (gradient is no longer an unbiased estimate under a uniform distribution)
- To correct this, importance-sampling weights are applied during updates

$$w_i = \left(\frac{1}{N \cdot P(i)}\right)^{\beta}$$

  - Where N: size of the replay buffer $P(i)$: sampling probability of transition i
  - $\beta \in [0,1]$: controls the amount of correction (usually increased toward 1 during training)
- Each sample's TD error is then weighted by w_i in the loss:

$$L = \frac{1}{B}\sum_i w_i, \delta_i^2$$

CODING

TD3